

PROJETO INTEGRADOR – MODELAGEM DE SOFTWARE E ARQUITETURA DE SISTEMAS

INTEGRANTES DO PROJETO

- Yohann Correia Cury – 25028106
- Italo de Andrade Goes – 26028878
- Luiz Felipe Marques Gomes – 26029225
- Stephany Felix da Cunha Cavaleiro – 26029131

SUMÁRIO

1. Introdução
2. Design Sprint
3. Requisitos de Sistema
4. Casos de Uso
5. Diagrama de Classe
6. Arquitetura do Sistema

1. INTRODUÇÃO

1.1 Processo de Desenvolvimento de Software

O processo adotado no desenvolvimento deste projeto foi o modelo incremental, permitindo a construção do sistema em etapas, com validações contínuas e refinamento progressivo das funcionalidades.

1.2 Apresentação

Este projeto tem como objetivo o desenvolvimento de um aplicativo desktop para gerenciamento de uma plataforma de jogos educacionais.

A solução proposta visa suprir a carência de plataformas especializadas em jogos educativos que ofereçam funcionalidades completas de gerenciamento, como controle de usuários, biblioteca de jogos, progresso e avaliações.

O sistema será utilizado por estudantes, permitindo acesso organizado aos jogos e acompanhamento do desempenho individual.

2. DESIGN SPRINT – IDEIAÇÃO E PROTOTIPAÇÃO

2.1 Desafio

Desenvolver um sistema desktop capaz de gerenciar jogos educacionais, incluindo funcionalidades de cadastro, autenticação, biblioteca, progresso e avaliação.

2.2 Entender e Mapear

Identificou-se a necessidade de centralizar o acesso aos jogos educacionais, facilitando o controle do aprendizado e a experiência do usuário.

2.3 Ideação – Trilha do Usuário

1. Cadastro do usuário
2. Login no sistema
3. Acesso ao catálogo de jogos
4. Inclusão de jogos na biblioteca
5. Execução dos jogos
6. Acompanhamento de progresso
7. Avaliação dos jogos

2.4 Prototipagem

Protótipos foram definidos priorizando usabilidade, com interface simples, menus claros e navegação objetiva.

3. REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01 – Cadastro de Usuário

Função: Cadastro de usuário

Descrição: Permitir que o usuário se registre no sistema

Entradas: Nome, e-mail, senha

Fonte: Usuário

Saídas: Usuário cadastrado

Ação: Armazenar dados no banco de dados

RFS02 – Login de Usuário

Função: Autenticação

Descrição: Permitir acesso ao sistema

Entradas: E-mail e senha

Fonte: Usuário

Saídas: Acesso autorizado ou negado

Ação: Validar credenciais

RFS03 – Visualização de Jogos

Função: Exibir catálogo

Descrição: Mostrar jogos disponíveis

Entradas: Solicitação do usuário

Fonte: Sistema

Saídas: Lista de jogos

Ação: Consultar banco de dados

RFS04 – Biblioteca de Jogos

Função: Gerenciar biblioteca

Descrição: Adicionar jogos à biblioteca

Entradas: Seleção de jogo
Fonte: Usuário
Saídas: Jogo adicionado
Ação: Registrar vínculo usuário-jogo

RFS05 – Avaliação de Jogos

Função: Avaliação
Descrição: Permitir avaliação de jogos
Entradas: Nota e comentário
Fonte: Usuário
Saídas: Avaliação registrada
Ação: Salvar avaliação

RFS06 – Progresso do Usuário

Função: Acompanhar progresso
Descrição: Monitorar desempenho
Entradas: Dados de uso
Fonte: Sistema
Saídas: Progresso atualizado
Ação: Registrar evolução

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RNF01 – Usabilidade

Função: Interface
Descrição: Interface simples e intuitiva
Entradas: Interação do usuário
Fonte: Sistema
Saídas: Navegação facilitada
Ação: Aplicação de boas práticas de UI

RNF02 – Segurança

Função: Proteção de dados
Descrição: Garantir segurança das informações
Entradas: Dados do usuário
Fonte: Sistema
Saídas: Dados protegidos
Ação: Criptografia de senha

RNF03 – Desempenho

Função: Performance
Descrição: Respostas rápidas
Entradas: Requisições
Fonte: Sistema
Saídas: Tempo de resposta reduzido
Ação: Otimização do sistema

RNF04 – Tecnologia

Função: Plataforma
Descrição: Uso da linguagem C#
Entradas: Código-fonte
Fonte: Desenvolvimento
Saídas: Aplicação funcional
Ação: Implementação em C#

RNF05 – Persistência

Função: Armazenamento
Descrição: Uso de banco de dados
Entradas: Dados
Fonte: Sistema
Saídas: Dados armazenados
Ação: Inserção no banco

RNF06 – Manutenibilidade

Função: Estrutura
Descrição: Sistema modular
Entradas: Código
Fonte: Desenvolvimento
Saídas: Facilidade de manutenção
Ação: Separação em camadas

4. CASOS DE USO

Caso de Uso 1 – Cadastro de Usuário

Ator: Usuário
Fluxo: Inserir dados - Validar - Cadastrar

Caso de Uso 2 – Acessar Jogos

Ator: Usuário
Fluxo: Login - Visualizar catálogo - Selecionar jogo

Caso de Uso 3 – Avaliar Jogo

Ator: Usuário
Fluxo: Selecionar jogo - Inserir avaliação - Salvar

5. DIAGRAMA DE CLASSE

Classes:

- Usuário
- Jogo
- Biblioteca
- Avaliação

Relacionamentos:

- Usuário possui Biblioteca
- Biblioteca contém Jogos
- Usuário realiza Avaliações

6. ARQUITETURA DO SISTEMA

6.1 Visão Geral do Sistema

Nome do Sistema: Plataforma de Jogos Educacionais

Objetivo do Sistema:

O sistema tem como objetivo fazer o gerenciamento de uma plataforma desktop de jogos educacionais, onde os usuários conseguem se cadastrar, fazer login, visualizar os jogos disponíveis, adicionar jogos na biblioteca, acompanhar seu progresso e também avaliar os jogos que foram usados.

Público-Alvo:

O público-alvo do sistema são estudantes que utilizam jogos educacionais como uma forma de apoio nos estudos e no aprendizado.

Principais Funcionalidades:

- Cadastro de usuários;
- Login de usuários;
- Visualização dos jogos educacionais disponíveis;
- Adicionar jogos na biblioteca do usuário;
- Acompanhamento do progresso nos jogos;
- Avaliação dos jogos com nota e comentário.

6.2 Estrutura Geral do Sistema

O sistema foi pensado como uma aplicação desktop desenvolvida em C#, tendo telas para o usuário mexer no sistema, classes para organizar as informações e regras principais, e um banco de dados local para guardar os dados cadastrados.

A estrutura básica do sistema é formada por:

- **Telas do sistema:** usadas para cadastro, login, visualização de jogos, biblioteca, progresso e avaliações;
- **Lógica da aplicação:** parte responsável por processar as ações feitas pelo usuário e fazer o sistema funcionar corretamente;
- **Classes do sistema:** representam os elementos principais do projeto, como Usuário, Jogo, Biblioteca e Avaliação;
- **Banco de dados:** usado para armazenar usuários, jogos, bibliotecas, progresso e avaliações.

6.3 Tecnologias Utilizadas

As principais tecnologias utilizadas no projeto são:

- C#;
- Aplicação Desktop;
- Banco de dados local;
- Programação orientada a objetos;

- Ferramenta de desenvolvimento compatível com C#.

6.4 Componentes do Sistema

Componente	Responsabilidade
Tela de Cadastro	Permitir que o usuário crie uma conta colocando nome, e-mail e senha.
Tela de Login	Permitir que o usuário entre no sistema usando e-mail e senha cadastrados.
Tela Principal	Mostrar as principais opções do sistema e permitir a navegação.
Catálogo de Jogos	Exibir os jogos educacionais que estão disponíveis na plataforma.
Biblioteca do Usuário	Guardar e mostrar os jogos que foram adicionados pelo usuário.
Controle de Progresso	Registrar e apresentar o avanço do usuário nos jogos.
Avaliação de Jogos	Permitir que o usuário avalie os jogos com uma nota e comentário.
Banco de Dados	Armazenar as informações do sistema de forma persistente.
Classes do Sistema	Representar os dados principais do sistema, como Usuário, Jogo, Biblioteca e Avaliação.

6.5 Diagrama Simplificado

Representação simplificada da arquitetura do sistema:

Usuário

|

Telas do Sistema Desktop

|

Lógica da Aplicação em C#

|

Classes do Sistema

|

Banco de Dados Local

O diagrama mostra que o usuário primeiro interage com as telas do sistema. Depois essas telas mandam as informações para a lógica da aplicação, que usa as classes principais para tratar os dados e salvar no banco de dados local.

6.6 Organização de Arquivos e Pastas

A organização do projeto pode ser feita da seguinte forma:

/src

/telas

/model

/database

/services

/util

Descrição das pastas:

- **/telas:** onde ficam as telas de cadastro, login, catálogo, biblioteca, progresso e avaliação;
- **/model:** onde ficam as classes principais do sistema, como Usuário, Jogo, Biblioteca e Avaliação;
- **/database:** onde ficam os arquivos ou classes relacionadas ao banco de dados local;
- **/services:** onde ficam algumas regras de negócio e funcionalidades do sistema;
- **/util:** onde ficam recursos auxiliares usados pela aplicação.